COMPUTER NETWORKS AND NETWORK PROGRAMMING
– SKILL 1 –

ENCAPSULATION BUILDER

"Packet Header Overhead and Network Efficiency Using Wireshark "

Student: THOTA JHANA SRI (2520090121)

Experiment date: 11 November 2026

Tool: Wireshark 4.6.9

Capture interface: Wi-Fi (Windows laptop)

Evidence: Original packet captures and screenshots from the student's Wireshark session.

1. Aim

To capture network traffic and examine how Ethernet, IPv4, and TCP/UDP headers encapsulate application data; calculate the header overhead and payload efficiency of observed packets.

2. Objectives

• Capture packets on the active Wi-Fi interface using Wireshark.

• Identify frame, Ethernet, IPv4 and TCP/UDP header fields.

• Calculate total header overhead, overhead percentage and payload efficiency for one TCP and one UDP packet.

• Compare the measured results and preserve screenshots as evidence.

3. Background

Encapsulation is the addition of protocol-specific headers as data moves down the network stack. Ethernet provides link-layer addressing; IPv4 provides logical addressing and routing information; TCP provides reliable, connection-oriented transport, while UDP provides lightweight datagram transport. Headers consume bytes that are not application payload.

For the observed Ethernet II + IPv4 packets, the captured frame length equals Ethernet header + IPv4 header + transport header + transport payload. The Ethernet frame check sequence (FCS) was not included in these captures; calculations therefore use the displayed frame length, without adding an assumed

4 bytes.

4. Tools and Experimental Procedure

1. Installed Wireshark and its packet capture support, then opened Wireshark.
2. Selected the active Wi-Fi interface and started live capture.
3. Generated traffic by browsing websites and then stopped capture.
4. Applied display filters tcp, udp and tcp.len > 0 to locate useful samples.
5. Selected TCP frame 3439 and UDP frame 1; expanded Ethernet II, IPv4 and transport sections.
6. Recorded actual displayed packet lengths and computed overhead and efficiency.
7. Saved or retained screenshots as proof; the original capture should also be saved locally as a .pcapng file.

5. Packet Observations

Observed parameter	**TCP frame 3439**	**UDP frame 1**
Protocol / application	TCP / HTTPS (port 443)	UDP / mDNS (port 5353)
Source IPv4	10.250.17.162	10.250.31.146
Destination IPv4	172.64.155.209	224.0.0.251
Captured frame length	1454 B	295 B
Ethernet II header	14 B	14 B
IPv4 header length	20 B	20 B
IPv4 total length	1440 B	281 B
Transport header	20 B	8 B
Transport length incl. data	1420 B	261 B
Application/transport payload	1400 B	253 B

TCP evidence: the selected frame shows 1454 captured bytes, IPv4 total length 1440, IPv4 header length 20, TCP header length 20, and TCP segment payload

1400. UDP evidence: frame 1 shows 295 captured bytes, IPv4 total length 281, UDP length 261, and UDP payload 253.

—----------

6. Calculations

6.1 Formulae

Header overhead (bytes) = Ethernet header + IPv4 header + TCP/UDP header Payload efficiency (%) = Transport payload ÷ Captured frame length × 100 Overhead (%) = Header overhead ÷ Captured frame length × 100

6.2 TCP frame 3439

Header overhead = 14 + 20 + 20 = 54 bytes

Frame size check = 54 + 1400 = 1454 bytes

Payload efficiency = (1400 ÷ 1454) × 100 = 96.29%

Header overhead percentage = (54 ÷ 1454) × 100 = 3.71%

6.3 UDP frame 1

Header overhead = 14 + 20 + 8 = 42 bytes

Frame size check = 42 + 253 = 295 bytes

Payload efficiency = (253 ÷ 295) × 100 = 85.76%

Header overhead percentage = (42 ÷ 295) × 100 = 14.24%

—----------

7. Comparison and Interpretation

Measure	TCP sample	UDP sample
Payload	1400 B	253 B
Overhead	54 B	42 B
Efficiency	96.29%	85.76%
Overhead share	3.71%	14.24%

Although the UDP transport header is shorter (8 B versus 20 B), the selected UDP packet has lower payload efficiency because it carries a much smaller payload. The difference is due to the specific packet sizes; it does not establish that TCP is generally more efficient than UDP. These values represent two captured packets, not end-to-end network throughput.

---

8. Answers to the Seven Skill Questions

1. What is encapsulation?
The process of adding protocol headers (and sometimes trailers) to data as it passes down the network stack before transmission.

2. What is the purpose of the Ethernet header?
It identifies the source and destination MAC addresses and indicates the payload type for local link delivery.

3. What information is stored in the IP header?
Among other fields, the source and destination IP addresses, version, header length, total length, time to live, and next-protocol identifier.

4. What is the difference between TCP and UDP headers?
TCP has a larger variable-length header with sequence/acknowledgment information and control flags; UDP uses a fixed 8-byte header with ports, length, and checksum.

5. Why does each protocol add a header?
Each layer needs control information for its role, such as local delivery, routing, or transport between applications.

6. What is packet overhead?
Bytes used for protocol control fields rather than application payload.

7. How does header overhead affect network efficiency?
A larger overhead fraction means a smaller fraction of the captured frame carries useful payload; efficiency also depends strongly on payload size.

---

9. Screenshot Evidence from the Laptop

The following images are taken from the Wireshark session used for the measurements. Packet 3439 and UDP packet 1 are identifiable in the capture.

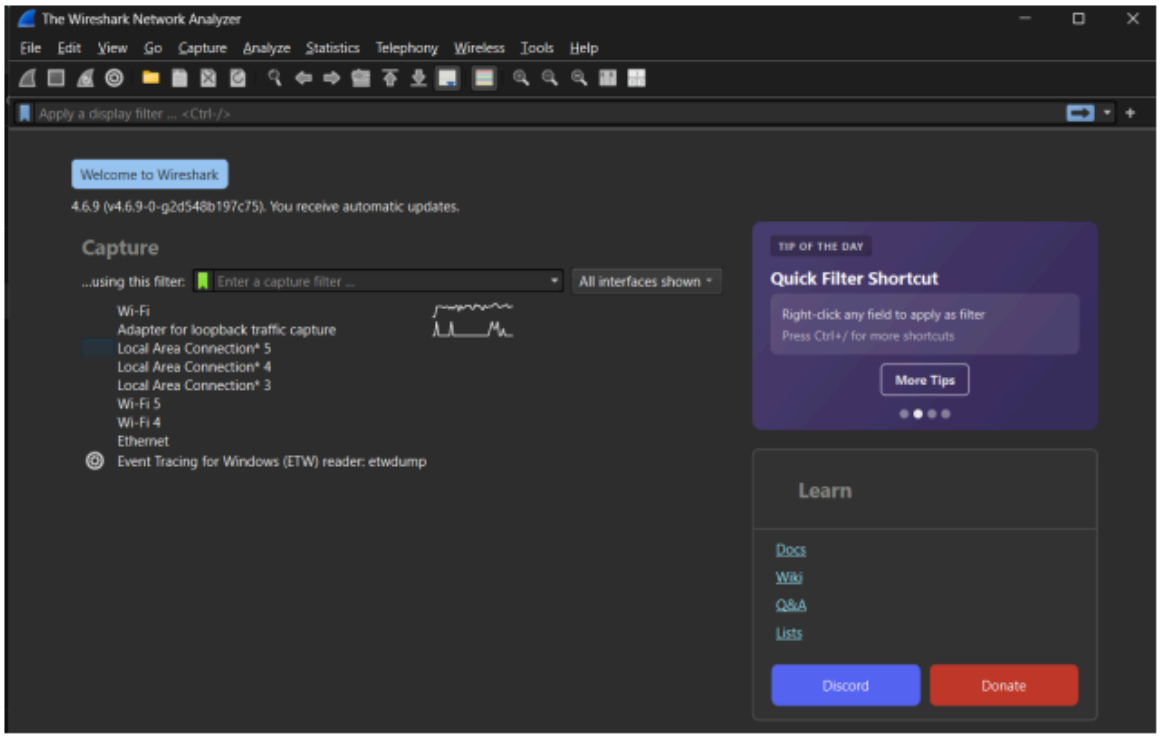


Figure 1. Wireshark installation verified; available Wi-Fi interface.

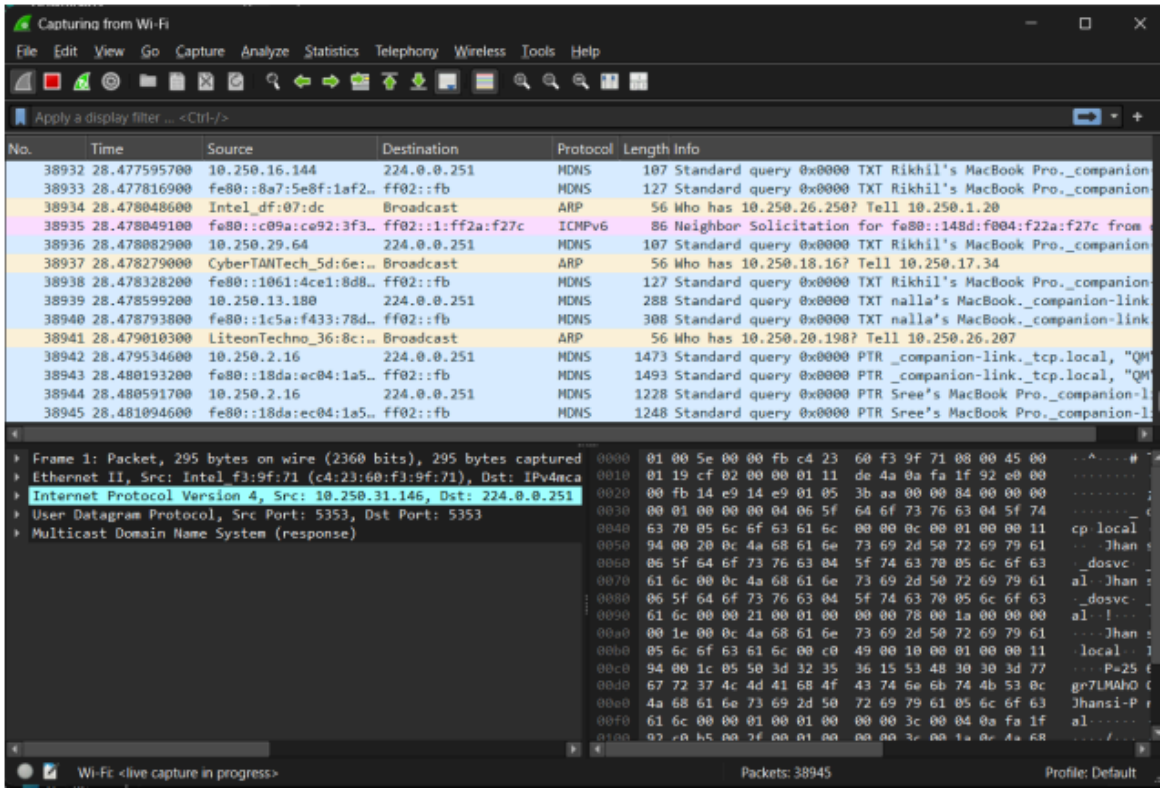


Figure 2. Live Wi-Fi capture showing network packets.

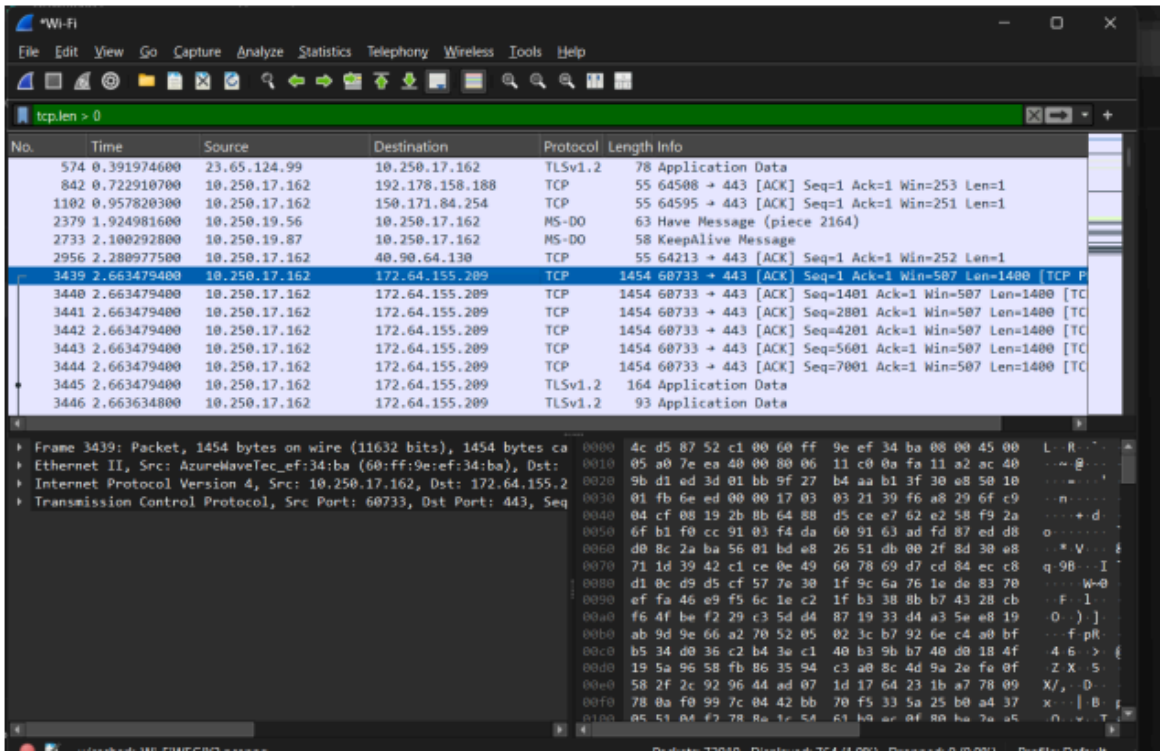


Figure 3. TCP frame 3439 selected; 1454-byte captured frame.

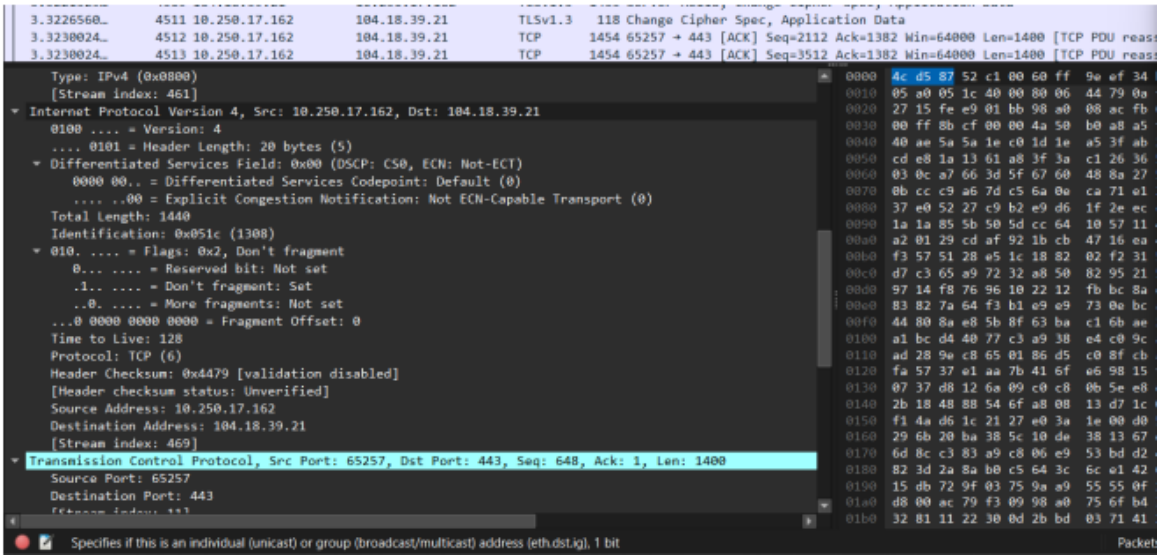


Figure 4. TCP IPv4 and TCP packet details (header lengths and payload).

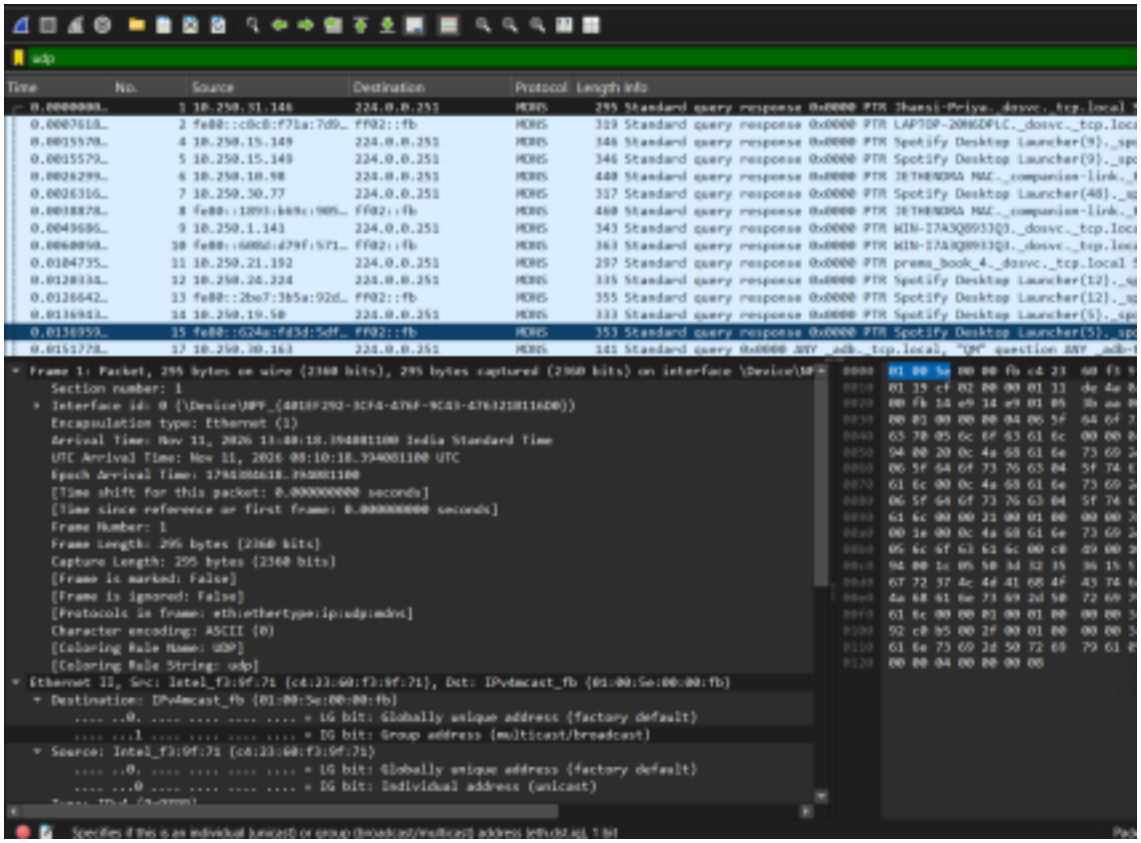


Figure 5. UDP filter and selected mDNS frame 1.

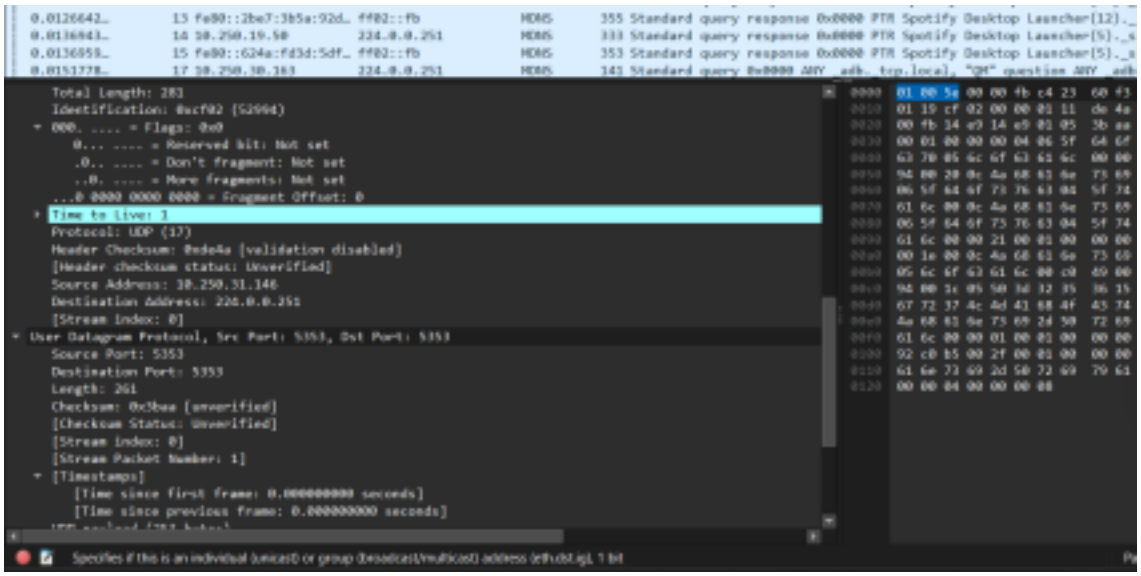


Figure 6. IPv4 and UDP header details for UDP frame 1.

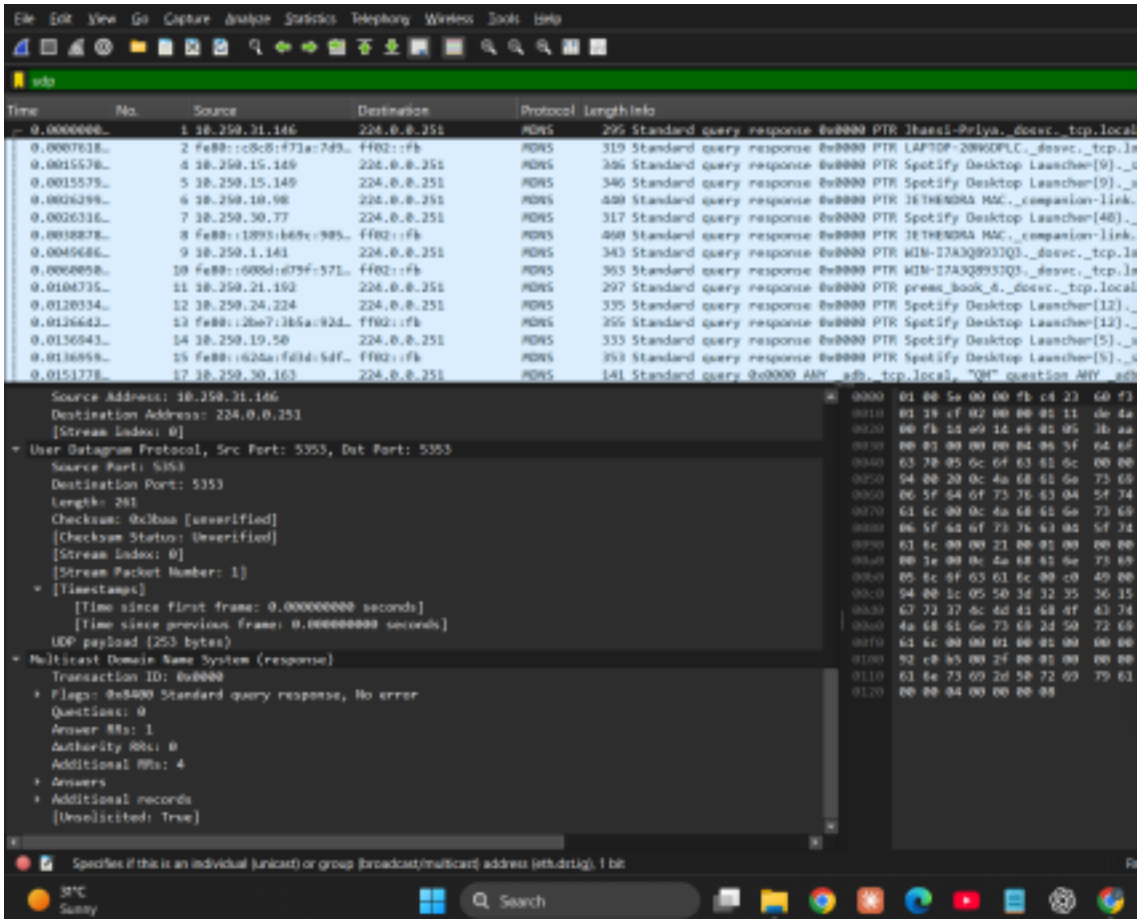


Figure 7. UDP length and 253-byte UDP payload.

10. Conclusion

Wireshark packet analysis confirmed encapsulation overhead in real captured traffic. The selected TCP frame carried 1400 bytes of payload with 54 bytes of headers, giving 96.29% payload efficiency. The selected UDP mDNS frame carried 253 bytes of payload with 42 bytes of headers, giving 85.76% payload efficiency. Payload size affects the observed efficiency, so conclusions should be restricted to these measured packets.